The Impact of Test-Time Compute in Post-Training

Thomas He, Kristine McLaughlin, Krithika Venkatasubramanian

Transitioning to Post-Training

Pre-Training (+ Training)

- Base knowledge through *lots* of data, large models
- Compute and memory requirements
- Parallelism (pipeline, data, tensor)

Post-Training

- Refinement process
- Fine-tune model with smaller, task-specific datasets
  - Supervised Fine-Tuning (SFT)
- Improve performance and delivery (social values, user pref.)

Today's Papers

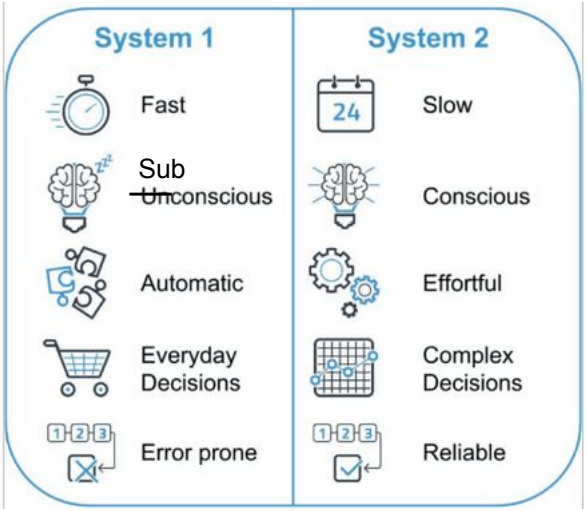
- Overall: improving reasoning capabilities in LLMs
  - Mathematical, scientific, coding
  - Large model sizes, need alternative solutions to current scaling methods
- Future of self-improving agents, AGI
- Effectiveness post-training methods
  - Achieve higher accuracy with tradeoffs
  - Scalability of success

Background

Test-Time Compute (TTC)

- System 1 vs. System 2 Thinking

Intuitive



Analytical

The amount of computation used when generating outputs

…

to improve performance on difficult tasks by encouraging more deliberate reasoning

Chain of Thought (CoT), 2022

- Model works through step-by-step reasoning process
- Thinking out loud

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

Systems:
Additional token generation
→ increase cost
Concise CoT (CCoT) reduce avg response length

TTC

- More complex methods, more compute
  - Reward modeling
  - Self verification
  - Search methods
  - Best-of-N sampling
  - STaR algorithm (RL inspired)
  - Verifier
- Benefits
  - More detailed and accurate reasoning
  - Alternative to traditional scaling approach
  - Smaller base models with performance more like large models

Current State, ~2024

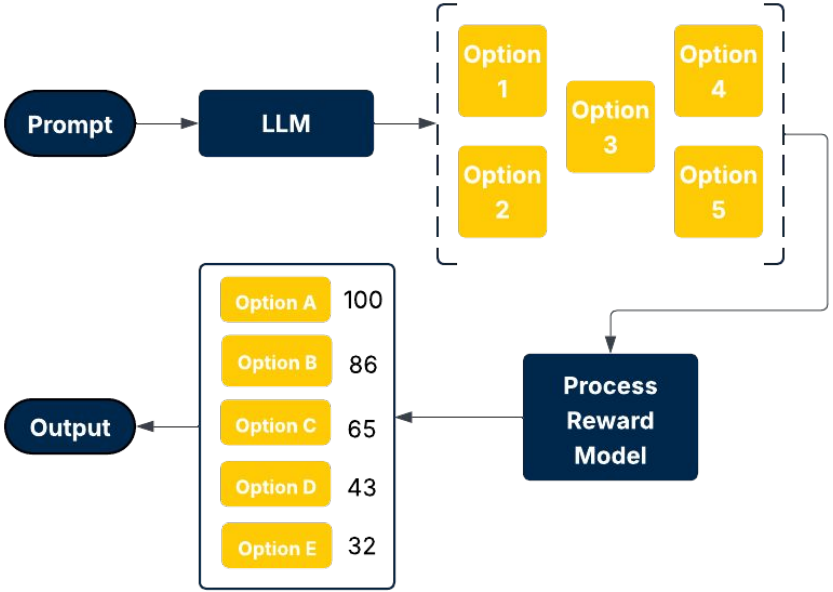
- Reasoning tasks: Effectiveness on complex tasks (math, proofs) improving but still limited reliability
- Inference-time computation: mixed results
  - Improves outputs, many experiments on CoT and deliberate problem solving
    - 4% → 74% on Game of 24 math puzzle
  - Don’t transfer well to complex tasks
  - LLMs still struggle to self-critique, advanced reasoning
- No analysis on different approaches for scaling TTC

Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

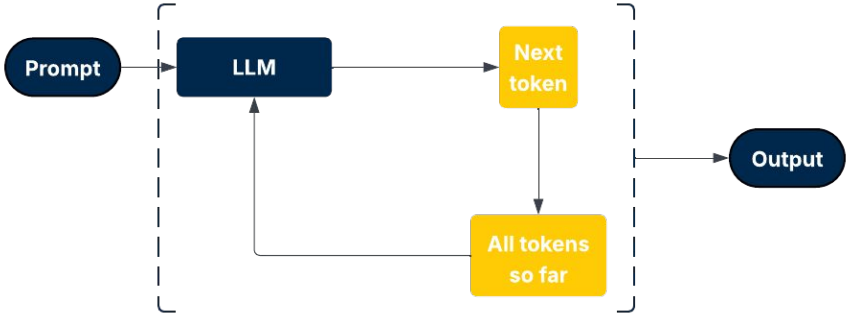
Google DeepMind, Aug. 2024

Test-Time Compute Methods

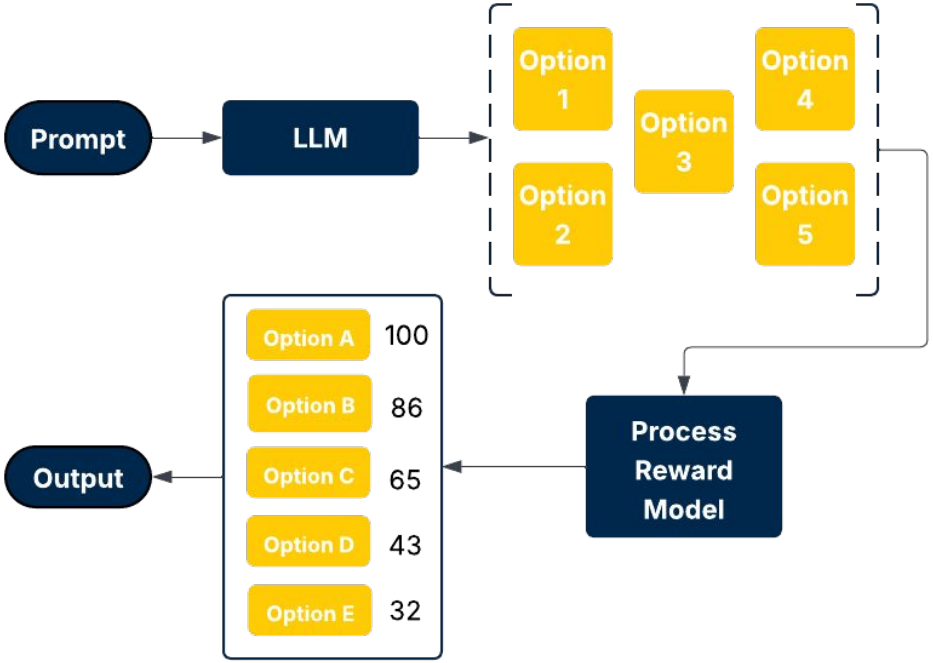
Method 1: Verifier



Method 2: Proposer/Revisor



Verifier Method



- LLM generates a set of possible outputs
  - **Granularity of outputs is dependent on the search algorithm in use**
- Output options are passed to a reward model, which assigns scores for each
- Scores are used to select the output

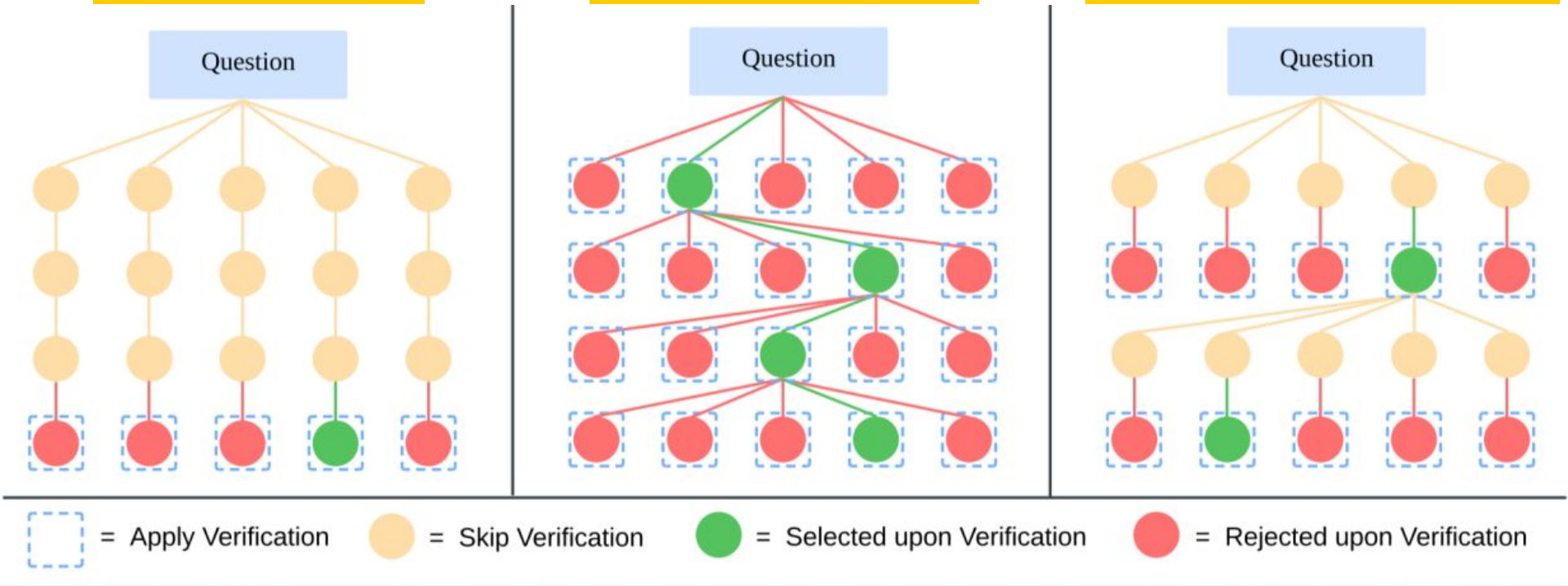
Question: How are the different output options generated?

Search Methods

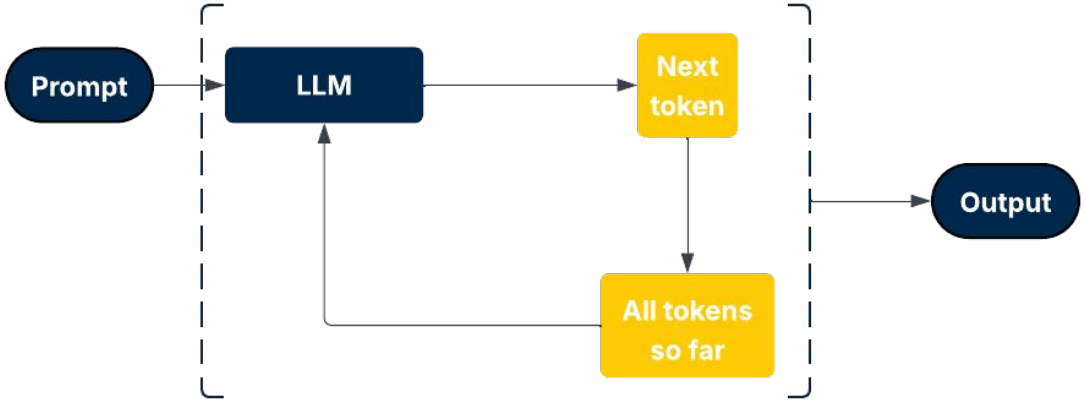
Method 1: Best-of-N

Method 2: Beam Search

Method 3: Lookahead Search

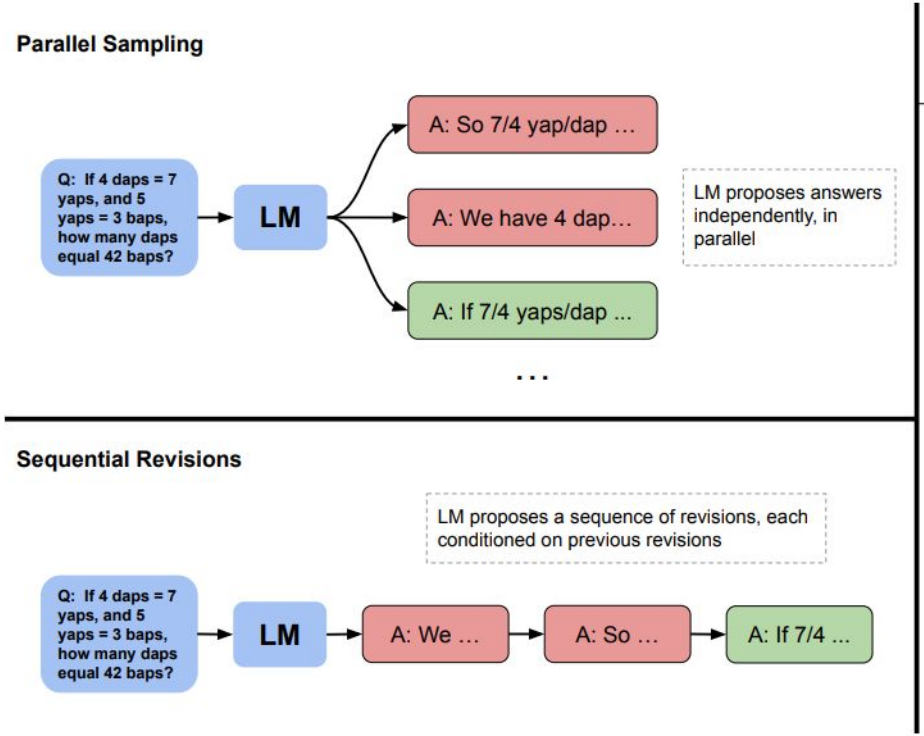


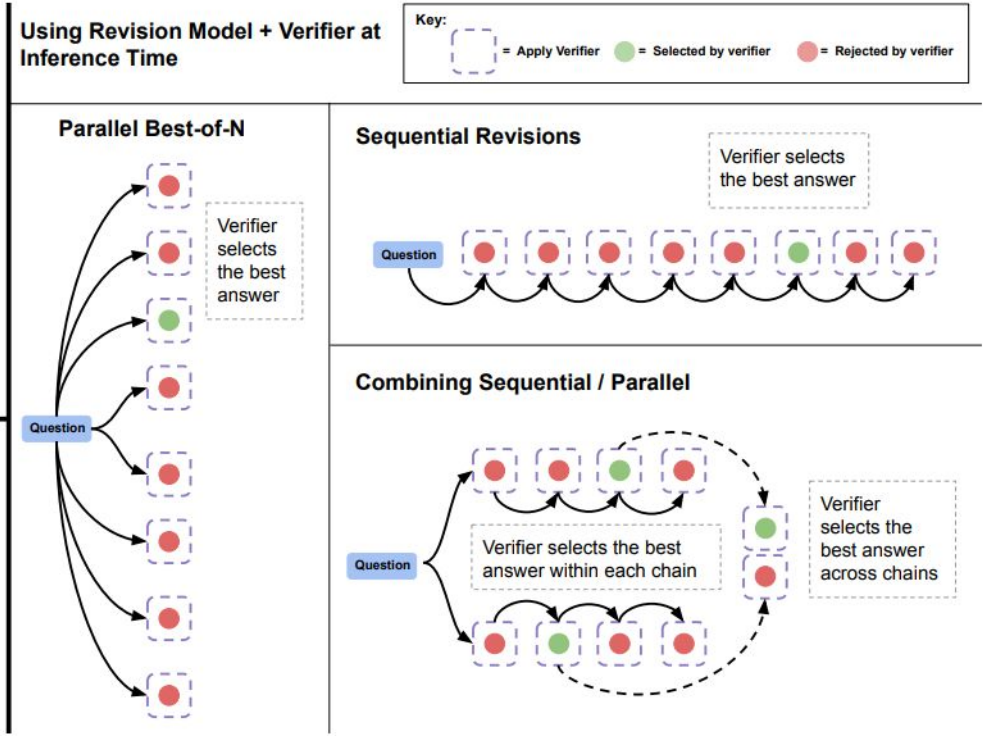
Proposer Method



- LLM is fine tuned to revise its own outputs
  - Some methods are inspired by reinforcement learning
- Tried to make existing outputs “better” in every iteration

Revision Methods





Scaling TTC “Optimally”

Question: How can we determine the most effective way to utilize TTC given a prompt and a TTC budget?

- **“Test-time compute-optimal strategy”:** Choose the highest performing strategy based on prompt difficulty
- Extra compute to assess question difficulty
  - **Oracle difficulty:** have a base LLM that does not use compute solve the problem multiple times, calculate pass rate
  - **Model-predicted difficulty:** spend a small amount of TTC compute budget to estimate difficulty
  - Paper argues negligible costs compared to performing TTC strategy

Experiments

MATH Dataset

High school competitive math problems

PaLM 2-S base model

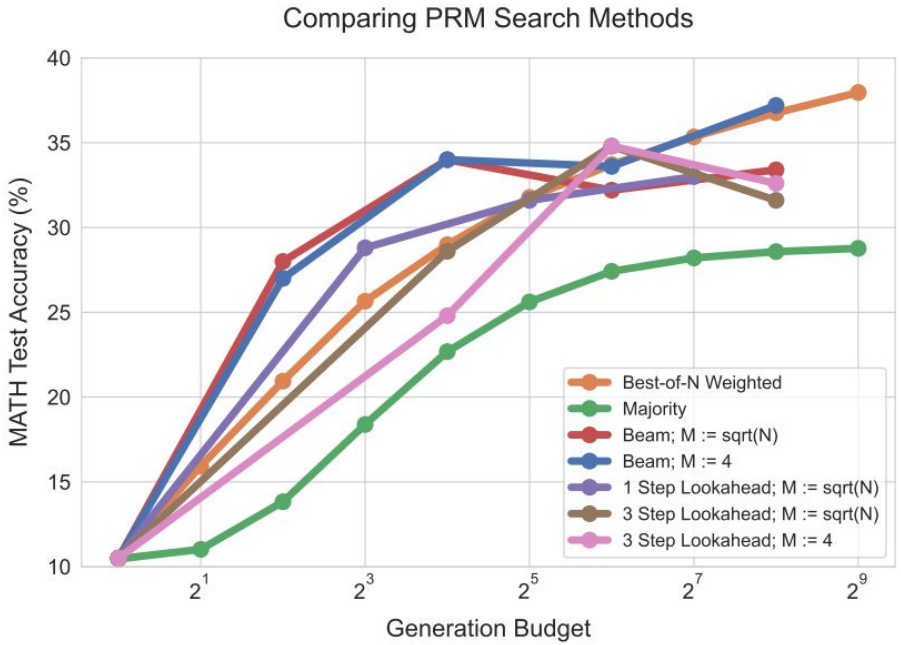
Representative of capabilities of contemporary LLMs, non-trivial performance on MATH

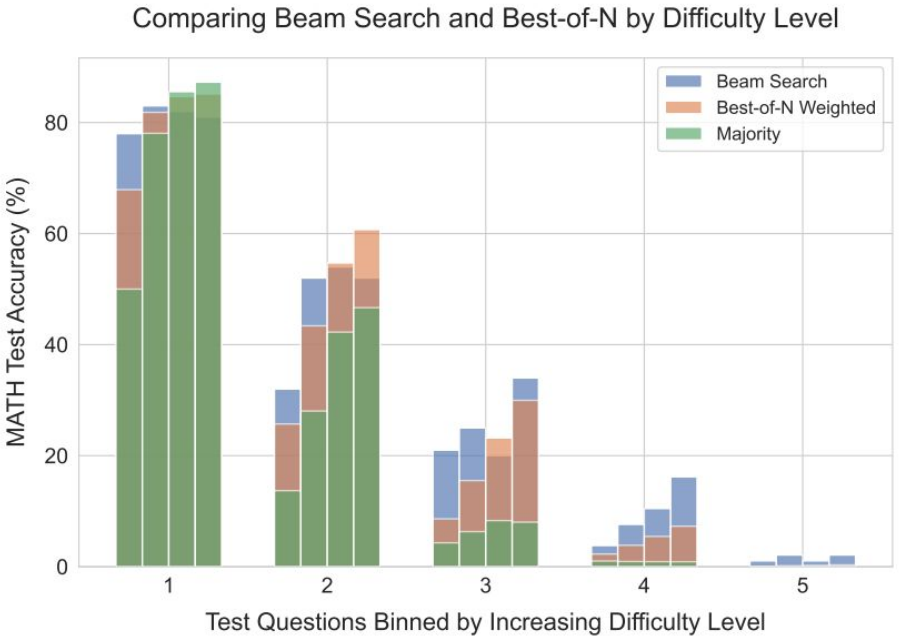
Experiment 1: Verifiers (PRMs)

Question: What is a compute-optimal scaling strategy (difficulty dependent) for search methods for verifiers?

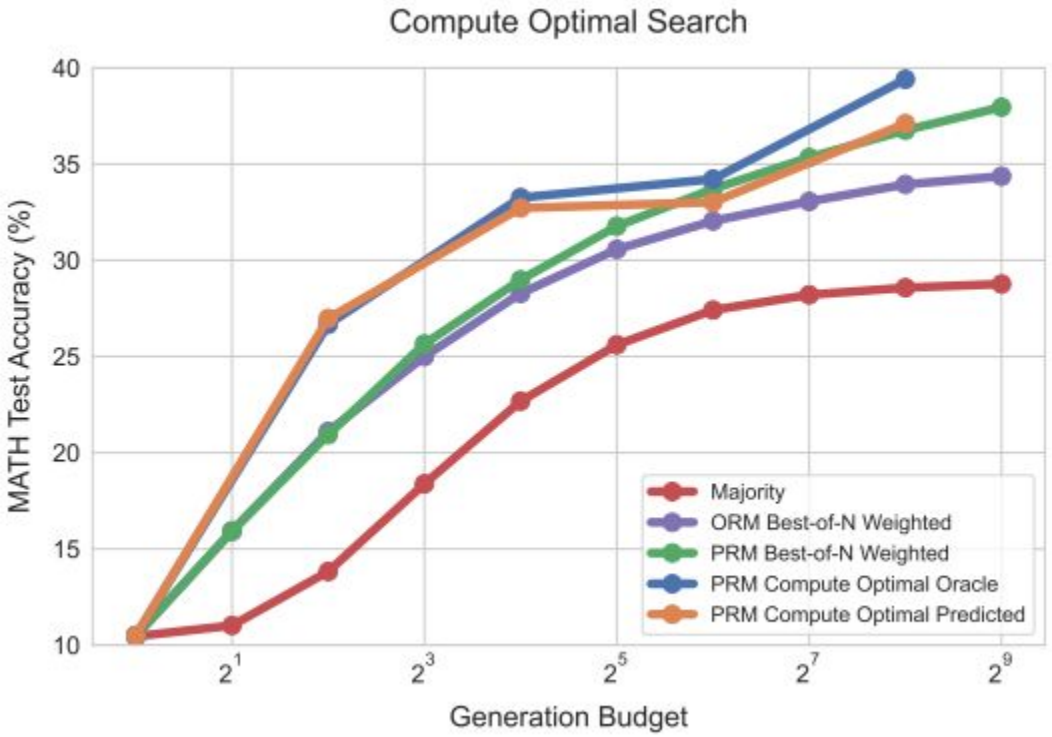
- Best-of-N weighted
- Beam search
  - Variable width: $\sqrt{N}$
  - Fixed width: 4
- Lookahead
  - Search 3 steps ahead
  - Search 1 step ahead
- Generation budget of up to 256: run searches enough to generate tokens equivalent to 256 full solutions

Verifier Results





Compute-Optimal Verifier



- Compute-optimal strategy: using the best possible search method for the difficulty of the question
- Generally, compute-optimal outperforms best-of-N

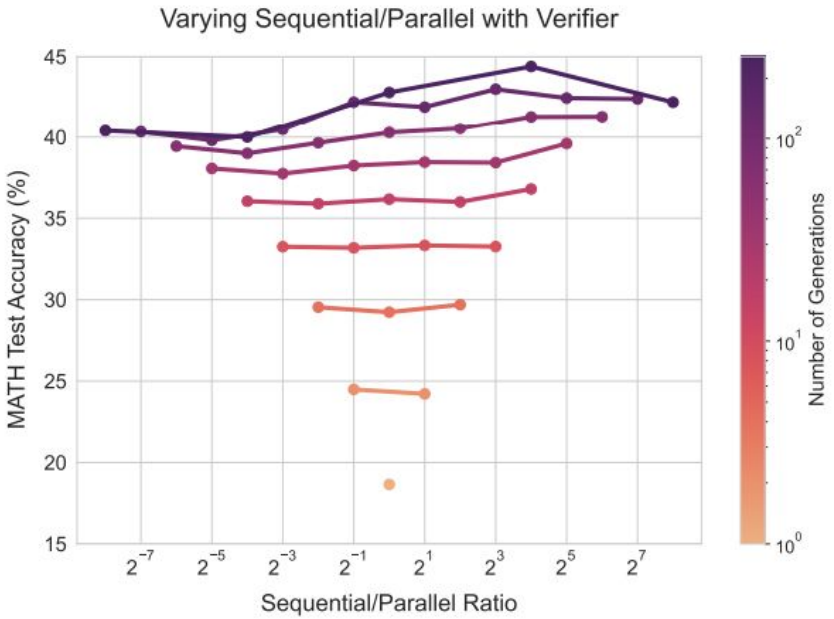
Systems takeaway: it's worth it to take the extra compute to figure out the difficulty of a question, because using the correct search algorithm for the job increases accuracy by quite a bit.

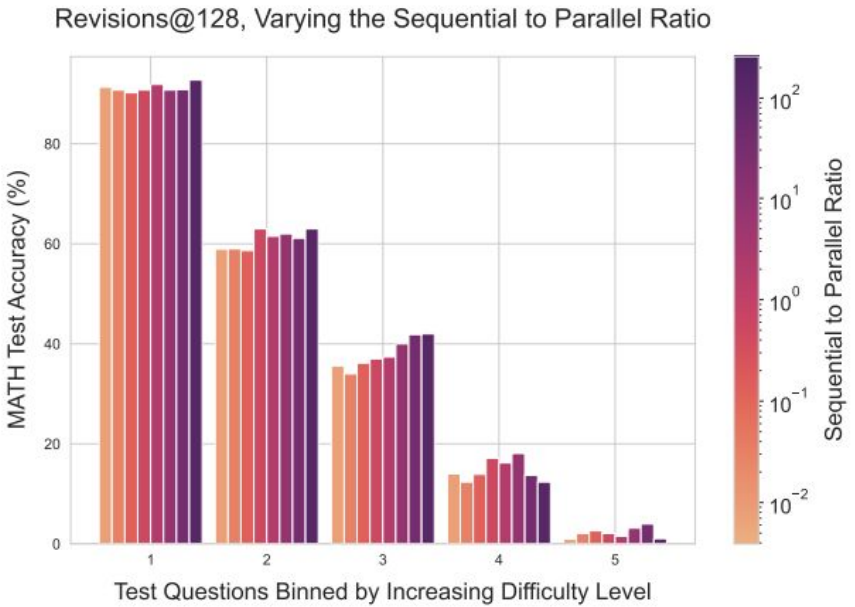
Experiment 2: Revisions

Question: What is a compute-optimal scaling strategy (difficulty dependent) for models to iterate on their own answers and revise their own proposal distributions?

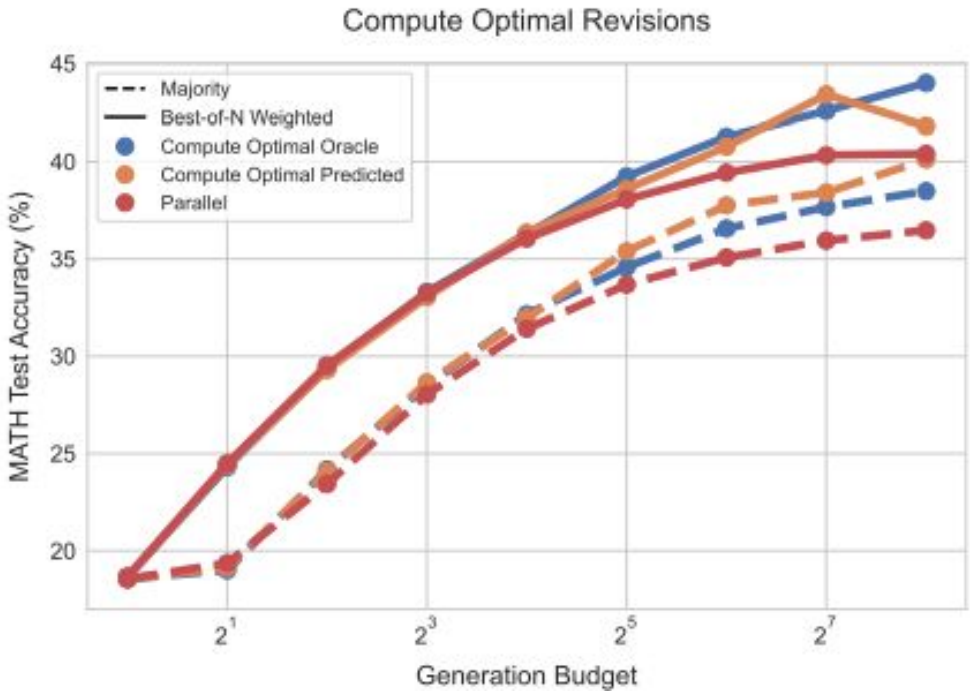
- Parallel: generate N independent answers at once and pick the best
- Sequential: generate an independent answer, then revise it N times
- Hybrid: generate a fixed number of parallel answers, then revise all of them in parallel

Revision Results





Revision Results



- Compute-optimal strategy: using the best possible ratio between sequential and parallel for the difficulty level of the question
- Generally, compute-optimal outperforms best-of-N

Systems takeaway: once again, it's worth it to take the extra compute to figure out the difficulty of a question, because striking the balance between sequential/parallel revision increases accuracy by quite a bit.

Experiment 3: Pre-training vs. TTC

Question: If we have the budget for more compute (FLOPs), should we spend it on increasing pretraining compute or additional TTC?

Same total compute

Small model + TTC

vs.

~14x larger model Standard inf

Note for graphs

- Ratio involved in figuring out how to match compute

$$R = \frac{D_{\text{inference}}}{D_{\text{pretrain}}}$$

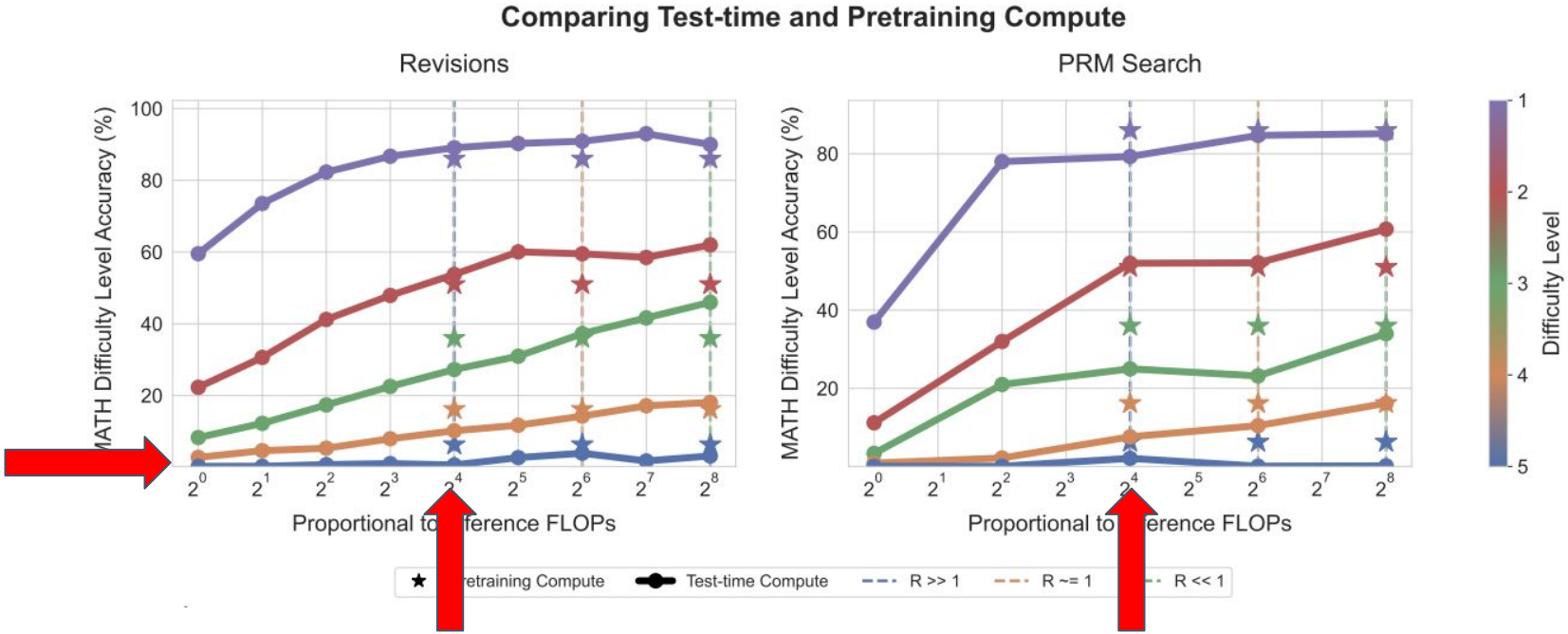
D_inference = # tokens generated at inference time
D_pretrain = # tokens used for pretraining

R >> 1

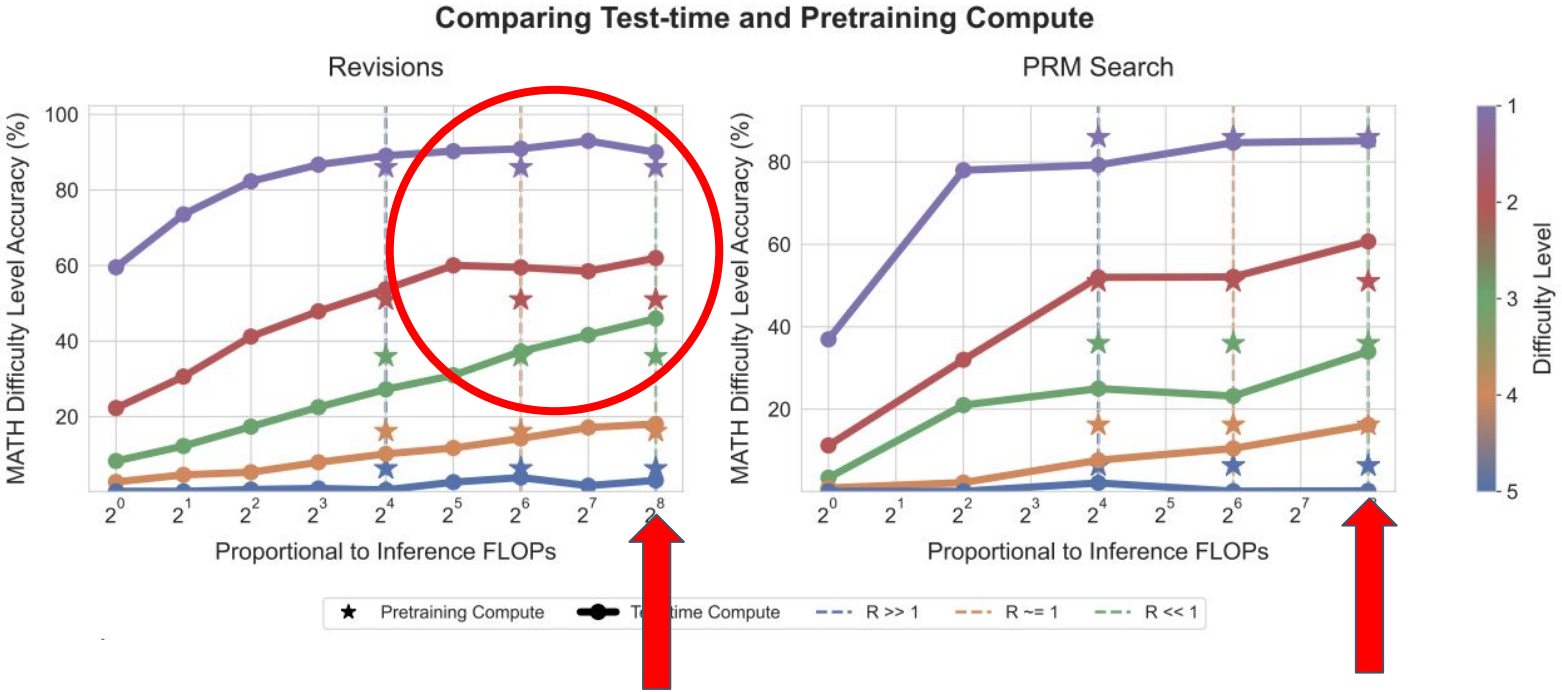
Large scale production settings, will see many more inference tokens over lifetime

R << 1

Self-improvement or TTC setups: model reasons more deeply on fixed eval set, higher pre-training cost



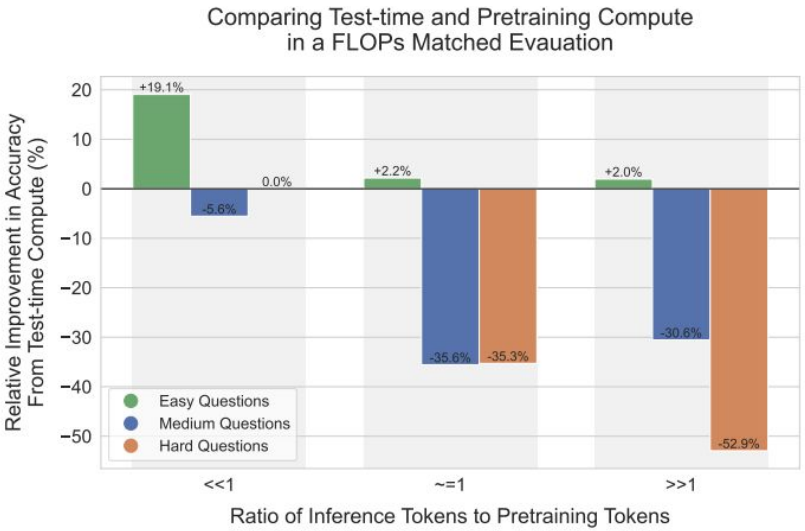
Very difficult questions or large inference load:
Star above line, more effective to have more pre-training
TTC cannot cover up for raw knowledge in pre-training



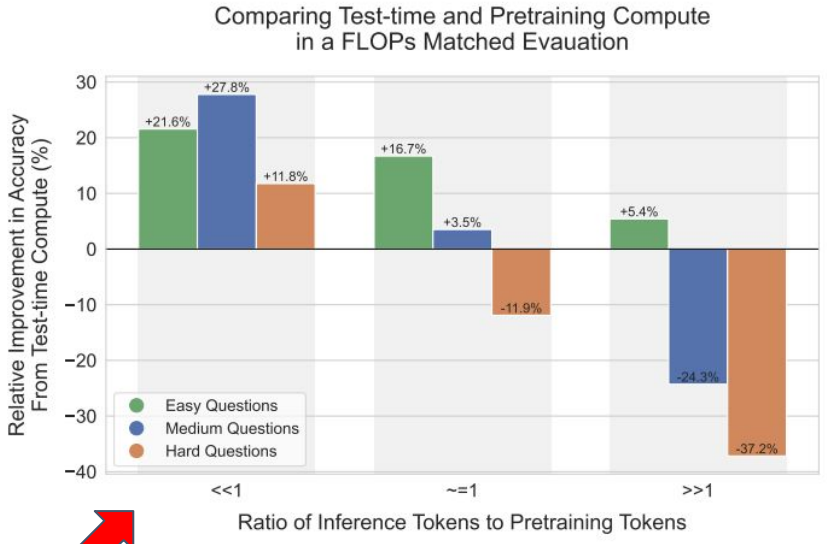
Easy/intermediate questions or lower inference load:
TTC outperforms! First time showing TTC gains over pretraining

Another visualization

Search against PRM verifier



Iterative revisions



Summary

- Difficulty of a question heavily impacts whether a certain approach works
- Compute-optimal approach: Accuracy can outperform best-of-N using up to **4x less compute**
- Smaller model with compute-optimal TTC approach can match or beat **performance of a 14x larger model**

Areas for Improvement

1. Further improving TTC scaling
    - More intertwining beyond PRMs and revisions
    - Not much improvement on hard problems
2. Assessing question difficulty quickly
    - Non-trivial amount of compute
    - Assessing difficulty vs. applying compute-optimal approach
3. Interleaving test time and training time compute
    - Use results of TTC to improve base model → iterative self-improvement loop

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

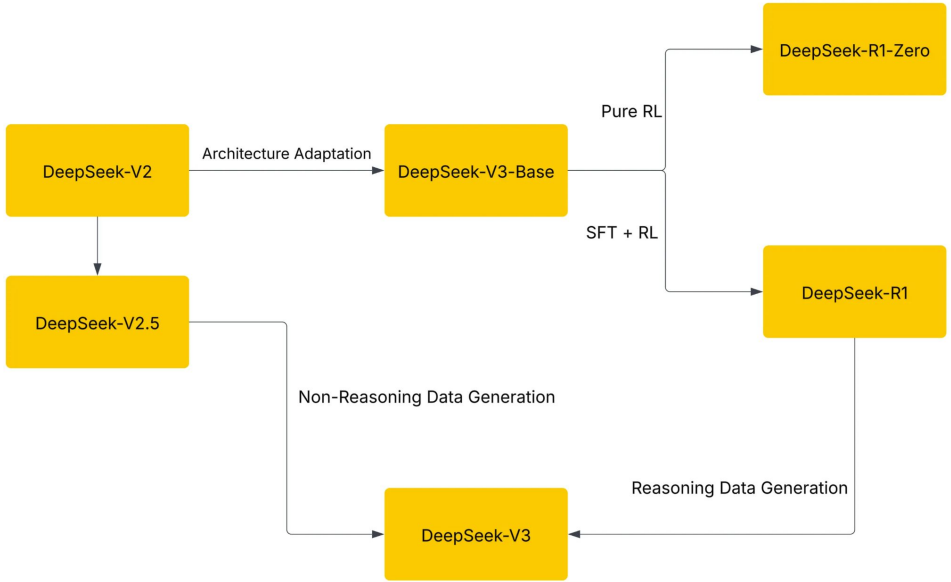
DeepSeek-AI, 2025

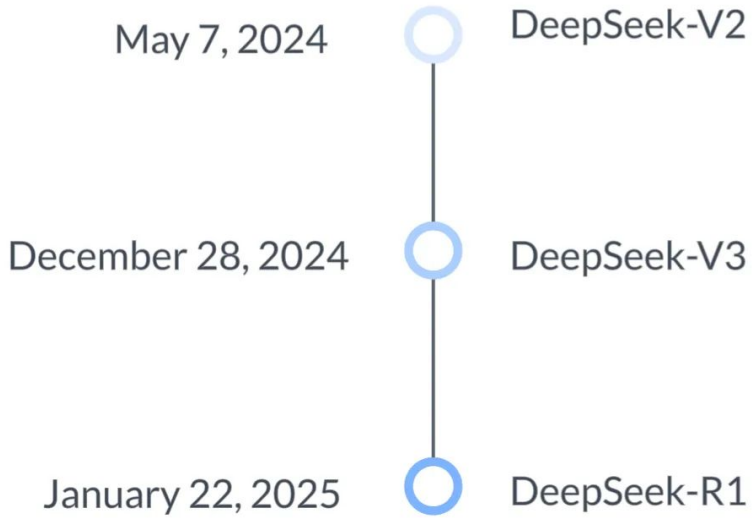
Background

Reinforcement Learning (RL)

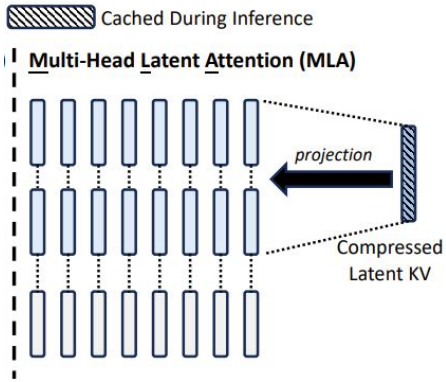
- Agent learns by making decisions in an environment and receiving rewards
- Goal: maximize reward
- Benefits
  - Discover own patterns/strategies
  - Often better than human specification, or when not possible
  - Robustness
  - Don’t need supervised data

The DeepSeek Family





Key Novelties



- V2 – Multi-Head Latent Attention
  - **93.3%** KV cache reduction on V2, **56.9%** on V3/R1
  - Increased max throughput generation by **5.76x**
- V3 – Novel pipeline parallelism design for Pre-training
- R1 – Reasoning-focused RL method for Post-training

Estimated 5x cheaper to train V3 than GPT4

						(40% of TCO)	(13% of TCO)	(16% of TCO)	(12.5% of TCO)	(9.5% of TCO)	(9% of TCO)	
Company	Model	Release	Nvidia GPU Gen	GPU Cluster Count	$ ea GPU	$ GPUs	$ Networking	$ Colocation	$ Cost of Capital	$ Misc - Eqp/Hardware/Other	$ Electricity	cluster TCO
OpenAI	GPT4	Aug 22	A100	25,000	$8,000	$200 M	$65 M	$80 M	$63 M	$48 M	$45 M	**$500 M**
Deepseek	V3	Dec 24	H800	2,048	$22,000	$45 M	$15 M	$18 M	$14 M	$11 M	$10 M	**$113 M**

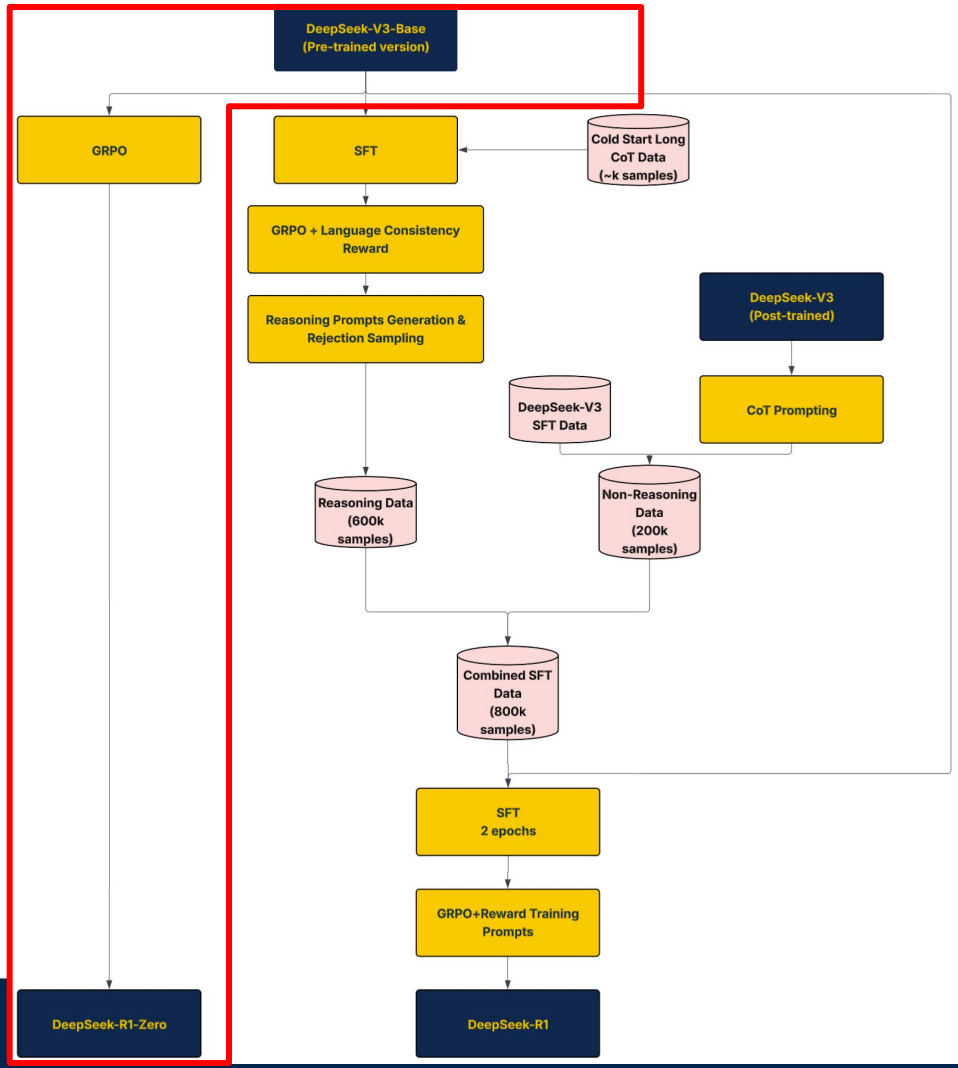
Now Let’s Get Into The Paper :)

Current Challenges

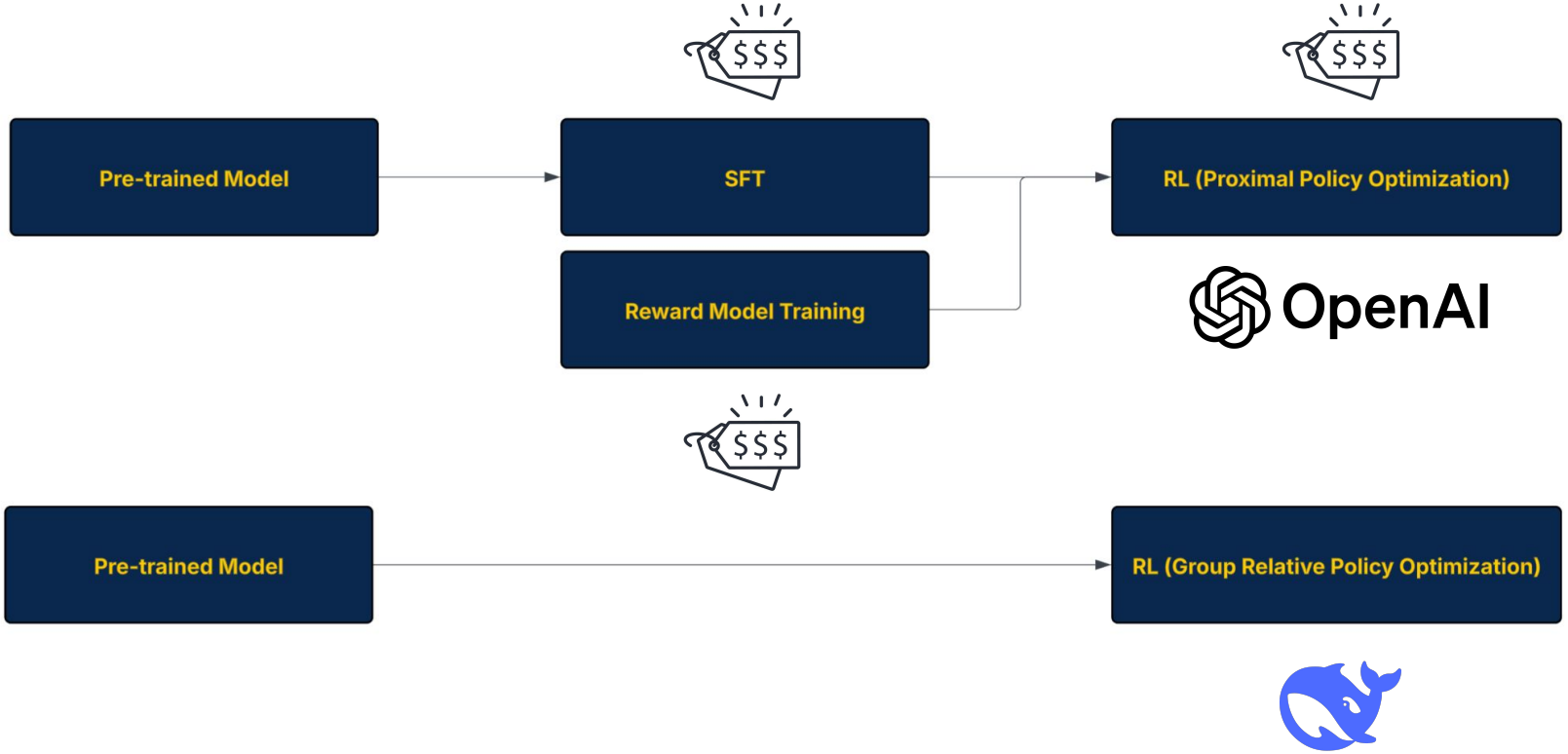
- GPT-o1 (Dec 2024)
  - Inference-time scaling by increasing length of CoT
  - Closed Source :(
- Generally thinking longer doesn’t always improve reasoning for very complex tasks
- Pure RL for reasoning tasks hasn’t been fully explored
  - Most use SFT + RL

R1 Motivations

- Can a no nonsense RL approach improve reasoning capability?
- Without the expensive human data, can we achieve o1-level reasoning performance with open-source models?


DeepSeek-V3-Base
(Pre-trained version)
GRPO
SFT
Cold Start Long CoT Data (~k samples)
GRPO + Language Consistency Reward
Reasoning Prompts Generation & Rejection Sampling
DeepSeek-V3 (Post-trained)
DeepSeek-V3 SFT Data
CoT Prompting
Reasoning Data (600k samples)
Non-Reasoning Data (200k samples)
Combined SFT Data (800k samples)
SFT
2 epochs
GRPO+Reward Training Prompts
DeepSeek-R1-Zero
DeepSeek-R1

Traditional Post-Training→Pure RL



Ruled-Based Reward Model

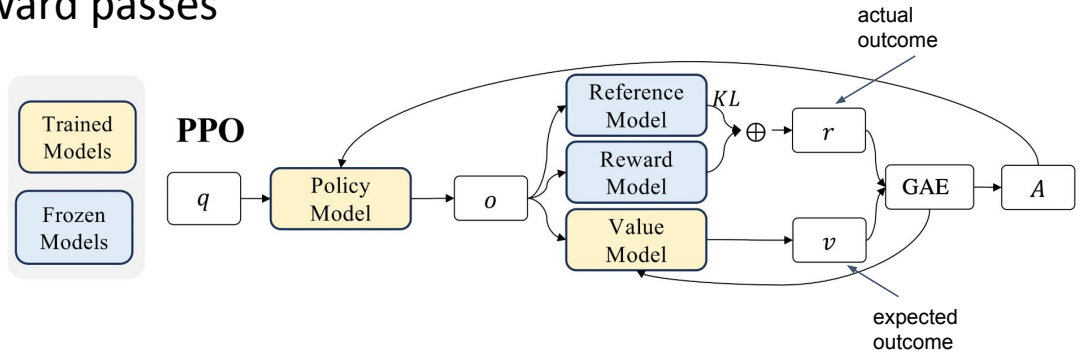
- Accuracy reward
  - Math & coding problems
- Format reward
  - Encourage model to put CoT into the think tags

Proximal Policy Optimization (PPO)

- Policy & Value model trained at the same time

Workflow:

1. Rollout (data collection)
   - **N** parallel instances of Policy model each running for **T** timesteps
   - Calculate Advantages
2. Optimization (model update)
   - Policy & Value backward passes

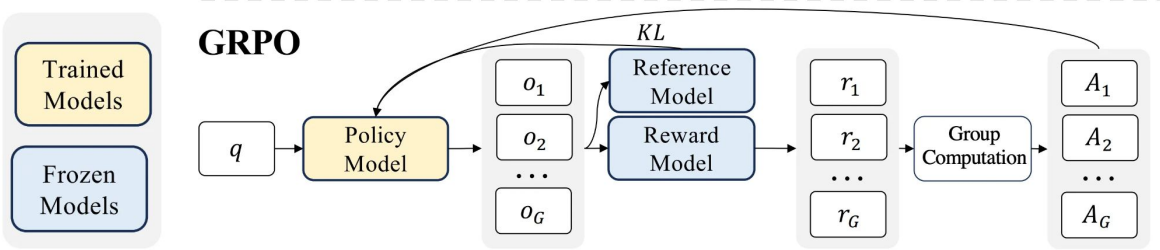


OpenAI, 2017

Group Relative Policy Optimization (GRPO)

Workflow

1. Rollout (data collection)
   a. Generate a batch of outputs for each prompt
   b. Advantage calculations
2. Optimization (model update)
   a. Policy model is updated for a number of iterations for a batch of prompts

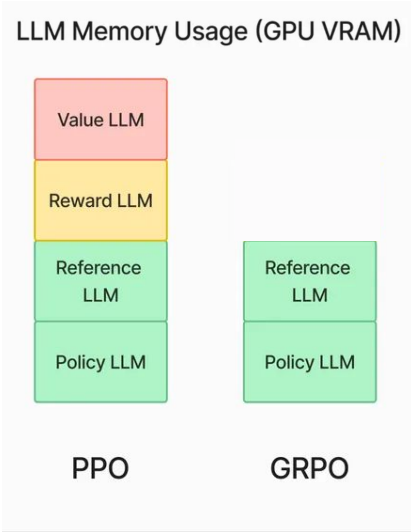


DeepSeekMath, 2024

PPO VS GRPO

VRAM: GRPO > PPO

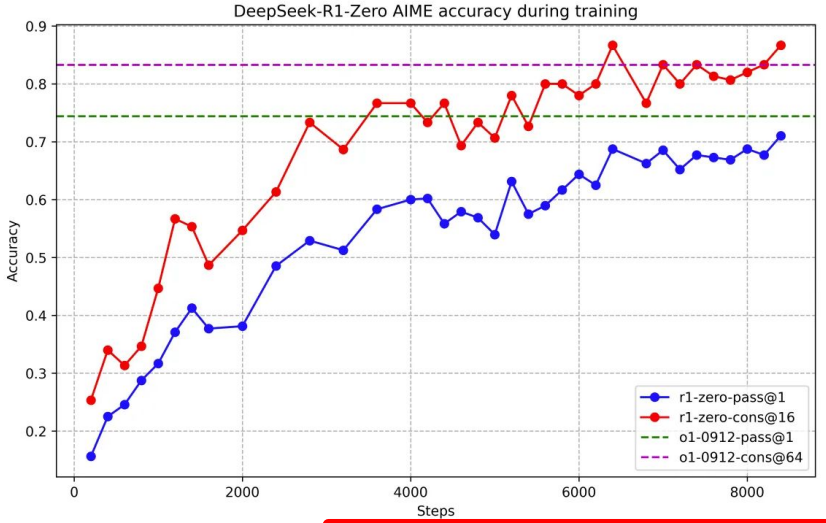
Computation Cost: Similar

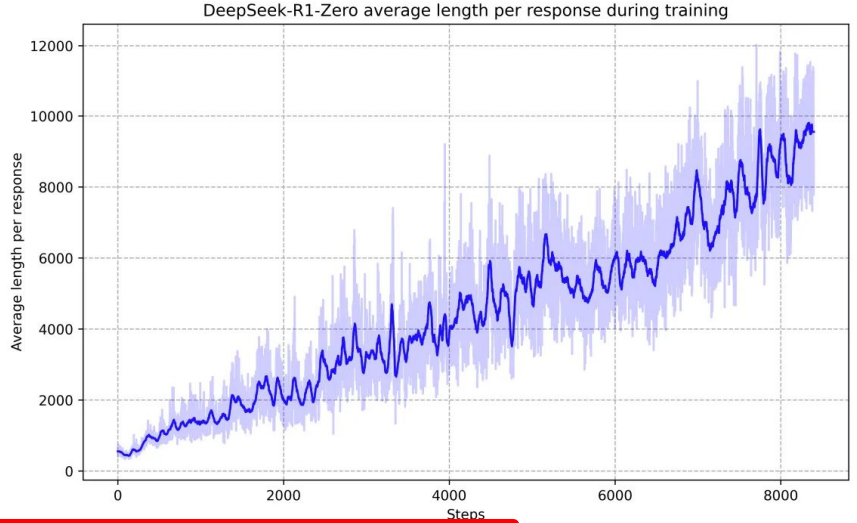


Model Size	Method	Time (hrs)	Memory (%)	KL $(\pi \| \| \pi_{\text{ref}})$
1.5B	*base*	/	/	/
	PPO	10.84	42.64	0.151
	GRPO	15.01	42.19	0.091
	REBEL	10.33	63.34	0.098
	A^*-PO	**6.92**	**41.59**	**0.069**
3B	*base*	/	/	/
	PPO	13.59	54.95	0.111
	GRPO	18.26	49.75	0.102
	REBEL	12.88	74.32	0.099
	A^*-PO	**8.78**	**49.28**	**0.082**
7B	*base*	/	/	/
	PPO	20.53	92.81	0.133
	GRPO	20.15	77.82	0.172
	REBEL	14.67	98.77	0.124
	A^*-PO	**11.01**	**76.57**	**0.078**

Gao, 2025

Result – R1-Zero





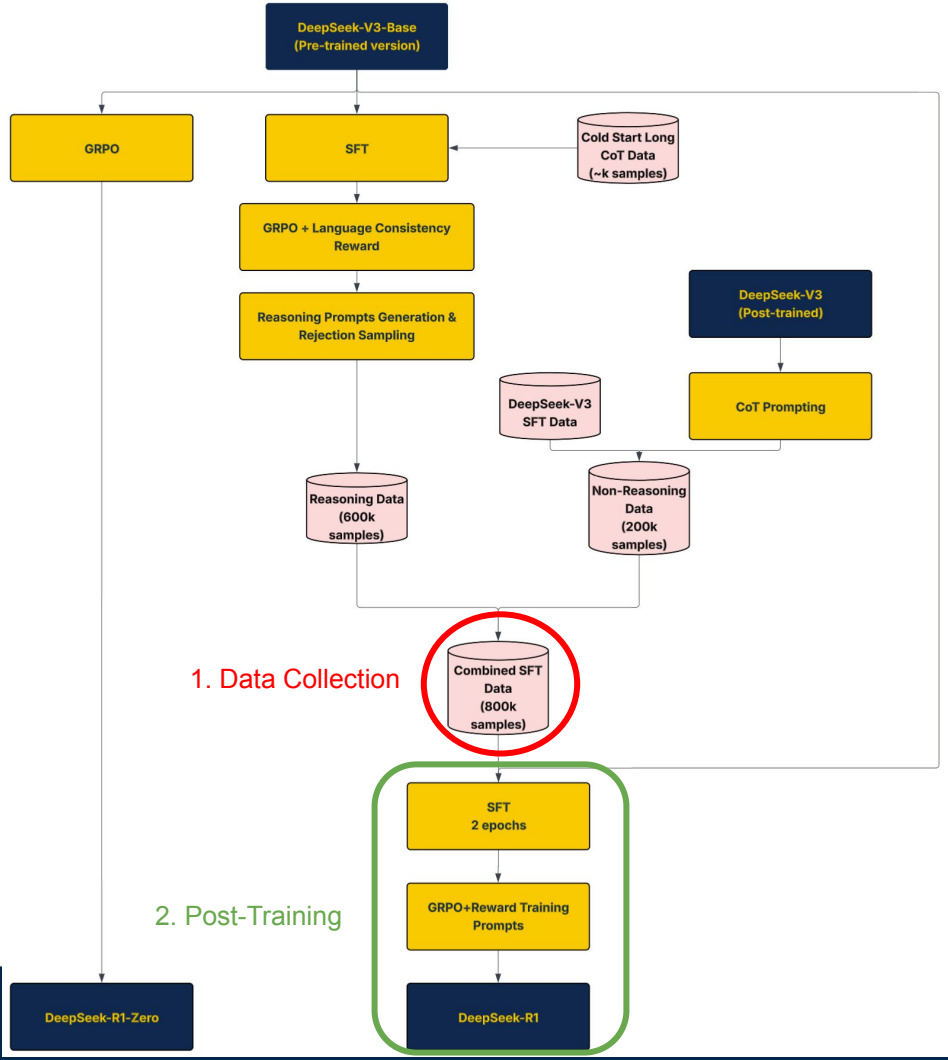
Systems takeaway: Proof that the model naturally acquires the ability to solve increasingly complex reasoning tasks by leveraging extended TTC.

R1-Zero Drawbacks

- Poor readability of the thinking
- Switch between different languages in a single response

Modifications in R1 Post-Training

- SFT using higher quality dataset
  - Help model learn high quality structured and well-reasoned data
  - Aiming to solve the readability/mixed-language issues
- RL via GRPO
  - Rule-Based RM
  - Model-Based RM

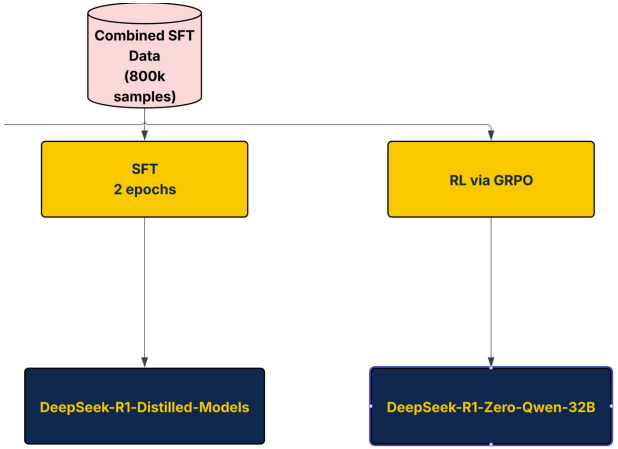

DeepSeek-V3-Base
(Pre-trained version)
GRPO
SFT
Cold Start Long CoT Data (~k samples)
GRPO + Language Consistency Reward
Reasoning Prompts Generation & Rejection Sampling
DeepSeek-V3 (Post-trained)
DeepSeek-V3 SFT Data
CoT Prompting
Reasoning Data (600k samples)
Non-Reasoning Data (200k samples)
1. Data Collection
Combined SFT Data (800k samples)
SFT 2 epochs
GRPO+Reward Training Prompts
2. Post-Training
DeepSeek-R1-Zero
DeepSeek-R1

DeepSeek-R1 Results

	Benchmark (Metric)	Claude-3.5-Sonnet-1022	GPT-4o 0513	DeepSeek V3	OpenAI o1-mini	OpenAI o1-1217	DeepSeek R1
	Architecture	-	-	MoE	-	-	MoE
	# Activated Params	-	-	37B	-	-	37B
	# Total Params	-	-	671B	-	-	671B
English	MMLU (Pass@1)	88.3	87.2	88.5	85.2	**91.8**	90.8
	MMLU-Redux (EM)	88.9	88.0	89.1	86.7	-	**92.9**
	MMLU-Pro (EM)	78.0	72.6	75.9	80.3	-	**84.0**
	DROP (3-shot F1)	88.3	83.7	91.6	83.9	90.2	**92.2**
	IF-Eval (Prompt Strict)	**86.5**	84.3	86.1	84.8	-	83.3
	GPQA Diamond (Pass@1)	65.0	49.9	59.1	60.0	**75.7**	71.5
	SimpleQA (Correct)	28.4	38.2	24.9	7.0	**47.0**	30.1
	FRAMES (Acc.)	72.5	80.5	73.3	76.9	-	**82.5**
	AlpacaEval2.0 (LC-winrate)	52.0	51.1	70.0	57.8	-	**87.6**
	ArenaHard (GPT-4-1106)	85.2	80.4	85.5	92.0	-	**92.3**
Code	LiveCodeBench (Pass@1-COT)	38.9	32.9	36.2	53.8	63.4	**65.9**
	Codeforces (Percentile)	20.3	23.6	58.7	93.4	**96.6**	96.3
	Codeforces (Rating)	717	759	1134	1820	**2061**	2029
	SWE Verified (Resolved)	**50.8**	38.8	42.0	41.6	48.9	49.2
	Aider-Polyglot (Acc.)	45.3	16.0	49.6	32.9	**61.7**	53.3
Math	AIME 2024 (Pass@1)	16.0	9.3	39.2	63.6	79.2	**79.8**
	MATH-500 (Pass@1)	78.3	74.6	90.2	90.0	96.4	**97.3**
	CNMO 2024 (Pass@1)	13.1	10.8	43.2	67.6	-	**78.8**
Chinese	CLUEWSC (EM)	85.4	87.9	90.9	89.9	-	**92.8**
	C-Eval (EM)	76.7	76.0	86.5	68.9	-	**91.8**
	C-SimpleQA (Correct)	55.4	58.7	**68.0**	40.3	-	63.7

R1 Model Distillation

1. SFT on select models
2. Large-scale RL on Qwen-32B



Able to empower small models with large-model like reasoning capability through SFT

Focus on systems challenges of large models

Discussion

- GRPO compute cost is comparable to PPO
- PRM and Monte-Carlo Tree Search did not work well
  - This contrasts with Paper 1!
- Distillation has given small models impressive reasoning capability
  - Less computation and better performance than RL on small models

Areas for Improvement

- Quantify performance in areas other than math/engineering/English/Chinese
  - Authors acknowledge that DeepSeek-R1 needs improvement in function calling, JSON output, etc.
- RL implementation is not efficient, results in very little improvement for DeepSeek-R1 vs DeepSeek-v3 for SWE
- Language mixing
  - R1 might use English for reasoning and responses even if query in different language

Thank You!

Appendix

TTC as Modifying Distribution

- Modify model's proposal distribution adaptively at test-time
  - Proposal distribution: method used to generate candidate outputs, probability distribution over words

Method 1: Output Level

Sample multiple candidates from standard LM, perform surgery to get ideal output

Method 2: Input Level

Augment given prompt with additional tokens to obtain a target distribution OR finetune to induce target distribution

How to define difficulty?

- 5 difficulty levels
  - Select best performing TTC strategy for each difficulty category independently
- Question difficulty = function of base LLM, use model's pass rate instead of hand-labeled difficulty bins
  - Oracle: ground-truth correctness checking
  - Model-predicted: averaged final answer score from a verifier

Scaling TTC “Optimally”

Question: How can we determine the most effective way to utilize TTC given a prompt and a TTC budget?

- **“Test-time compute-optimal strategy”:** Choose hyperparameters of a given test-time strategy for best performance on a given prompt
  - Maximize the accuracy of target distribution
  - Easier problems revisions vs harder problems different strategies
- Approximate compute-optimal strategy as a function of prompt difficulty
  - Difficulty in 5 levels, function of base LLM pass rate
  - Select best performing TTC strategy for each difficulty category independently

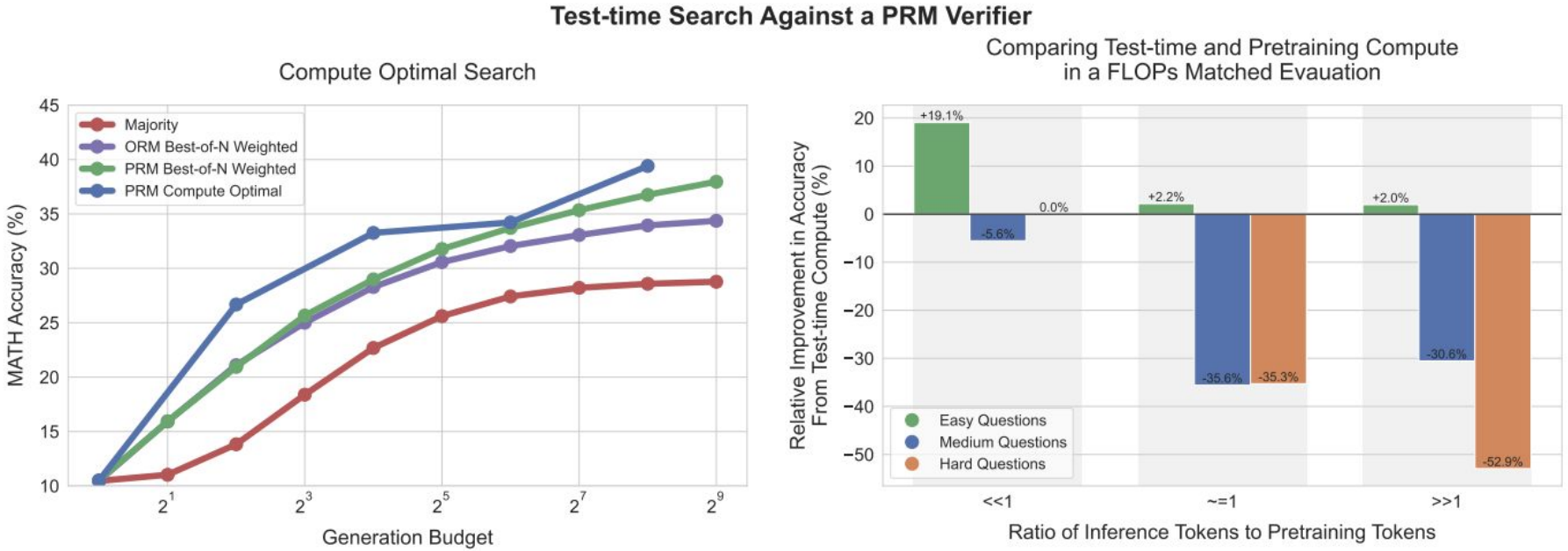
Finding TTC optimal scaling strategy

Hyperparameter selection:

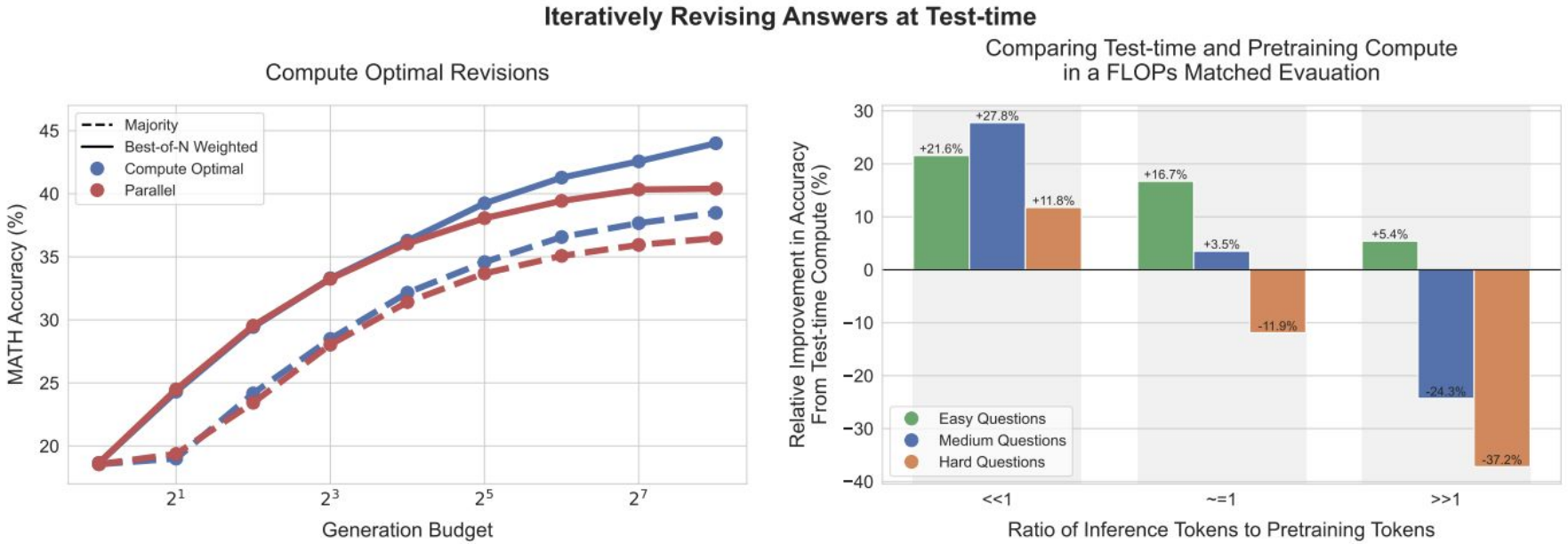
$$\theta^{*}_{q,a^{*}(q)}(N) = \text{argmax}_{\theta}\left(\mathbb{E}_{y\sim\text{Target}(\theta,N,q)}\left[\mathbb{1}_{y=y^{*}(q)}\right]\right),$$

Fill this slide out with what the different parts mean

Overall Results: Verifier



Overall Results: Revisions



PPO & GRPO

PPO Objective Function

Clipped PPO

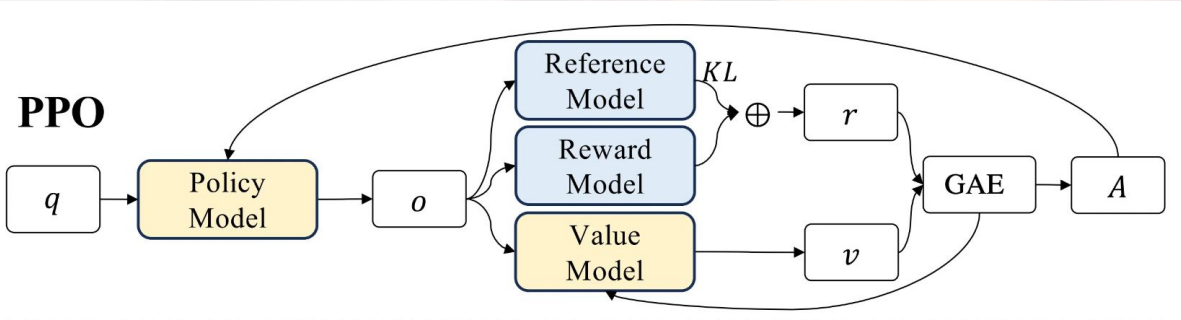
$$L_t(\theta) = r_t(\theta)\hat{A}_t - \beta\,\mathrm{KL}[\pi_{\theta_{\text{old}}}, \pi_\theta]$$

KL Penalty PPO

$$L_t(\theta) = \min(r_t(\theta)\hat{A}_t, \mathrm{clip}(r_t(\theta)), 1-\epsilon, 1+\epsilon)\hat{A}_t$$

t = time step

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$



GRPO Objective Function

$$r_i(\theta) = \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}$$

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_j\}_{j=1}^G)}{\text{std}(\{r_j\}_{j=1}^G)}$$

GRPO Objective for a group of outputs

$$\mathcal{L}_{\text{group}}(\theta; q) = \frac{1}{G} \sum_{i=1}^{G} \left[\min \left(r_i(\theta)\, \hat{A}_i,\ \text{clip}(r_i(\theta), 1-\epsilon, 1+\epsilon)\, \hat{A}_i \right) \right] - \beta\, D_{\text{KL}}\big(\pi_\theta(\cdot \mid q) \,\|\, \pi_{\text{ref}}(\cdot \mid q)\big)$$

From PPO *From PPO* ***New KL Divergence***

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\mathcal{L}_{\text{group}}(\theta; q) \right]$$

PPO Algorithm Pseudocode

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

GRPO Algorithm Pseudocode

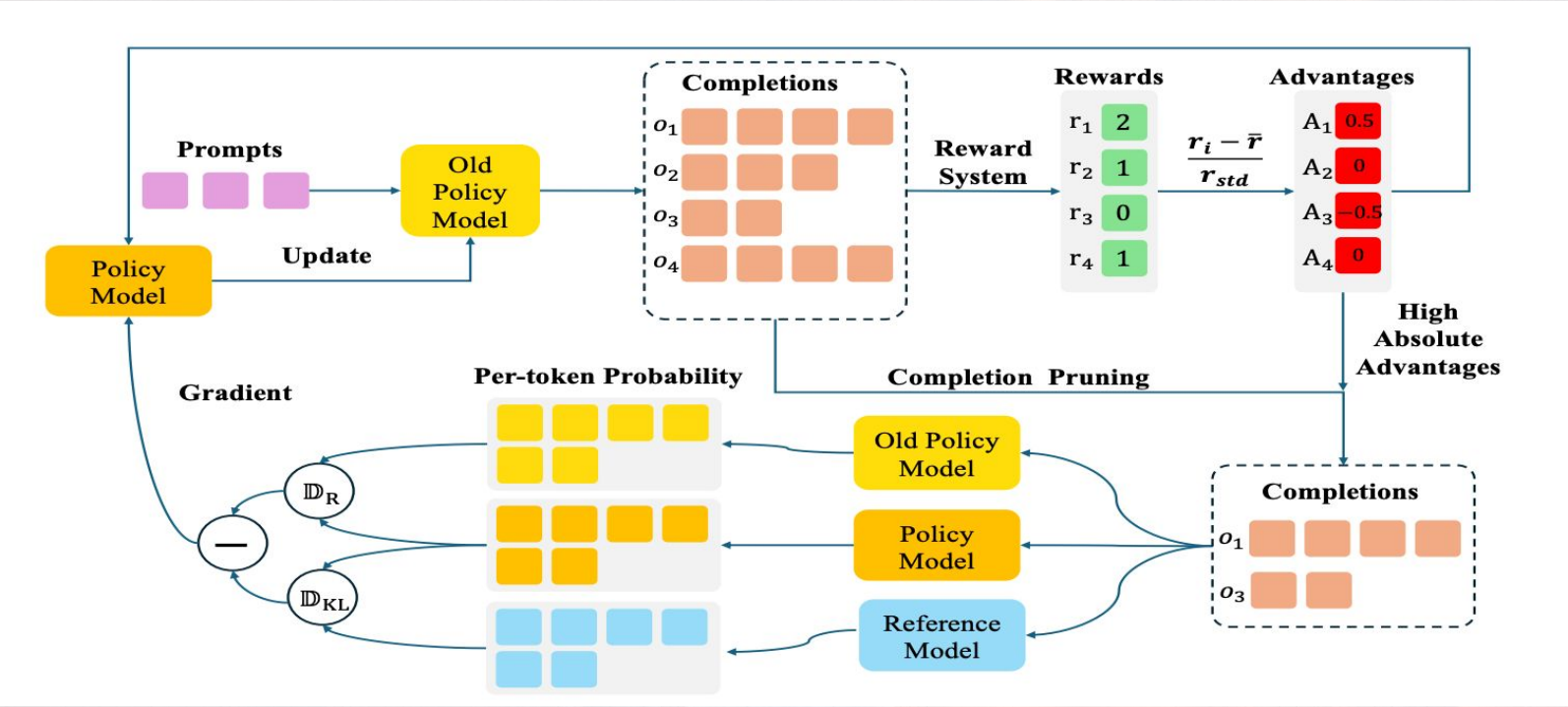
Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_φ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

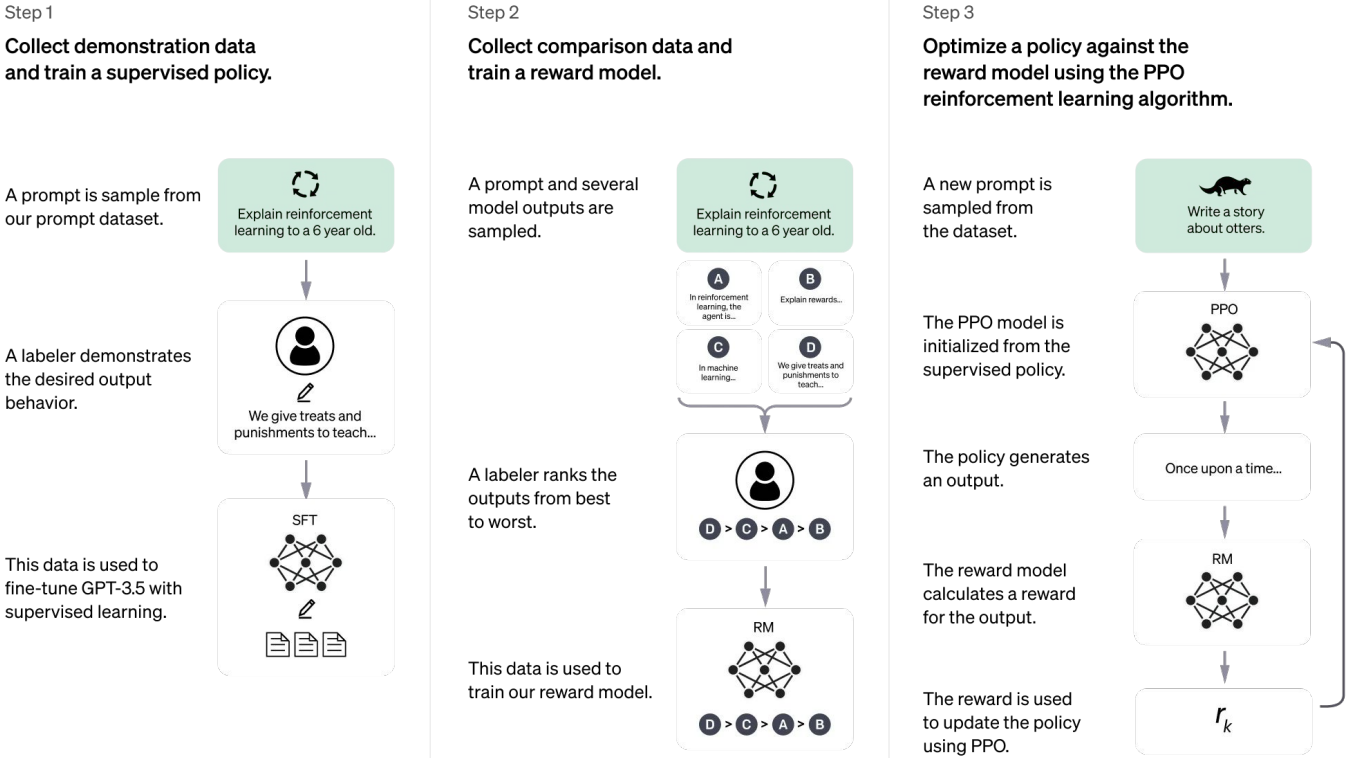
policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for iteration = 1, . . . , I **do**
    reference model $\pi_{ref} \leftarrow \pi_\theta$
    for step = 1, . . . , M **do**
        Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
        Update the old policy model $\pi_{\theta_{old}} \leftarrow \pi_\theta$
        Sample G outputs $\{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(\cdot \mid q)$ for each question $q \in \mathcal{D}_b$
        Compute rewards $\{r_i\}_{i=1}^{G}$ for each sampled output o_i by running r_φ
        Compute $\hat{A}_{i,t}$ for the t-th token of o_i through group relative advantage estimation.
        for GRPO iteration = 1, . . . , μ **do**
            Update the policy model π_θ by maximizing the GRPO objective (Equation 21)
    Update r_φ through continuous training using a replay mechanism.

Output π_θ

Completion Pruning Policy Optimization (CPPO)

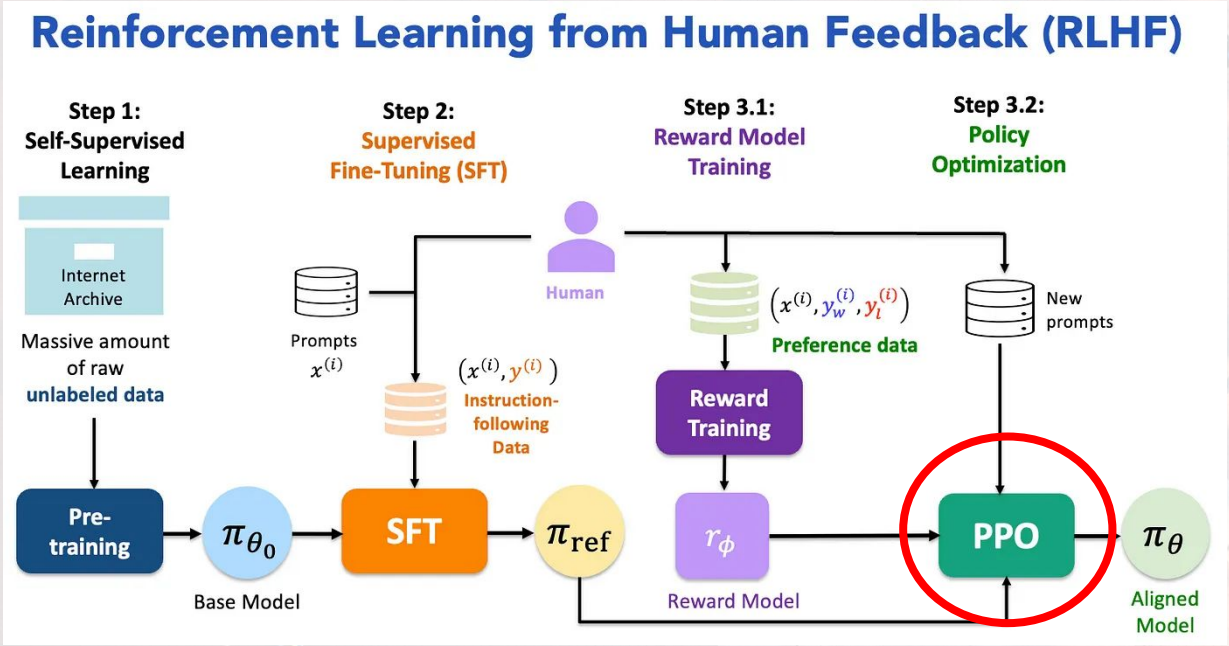


ChatGPT-1130



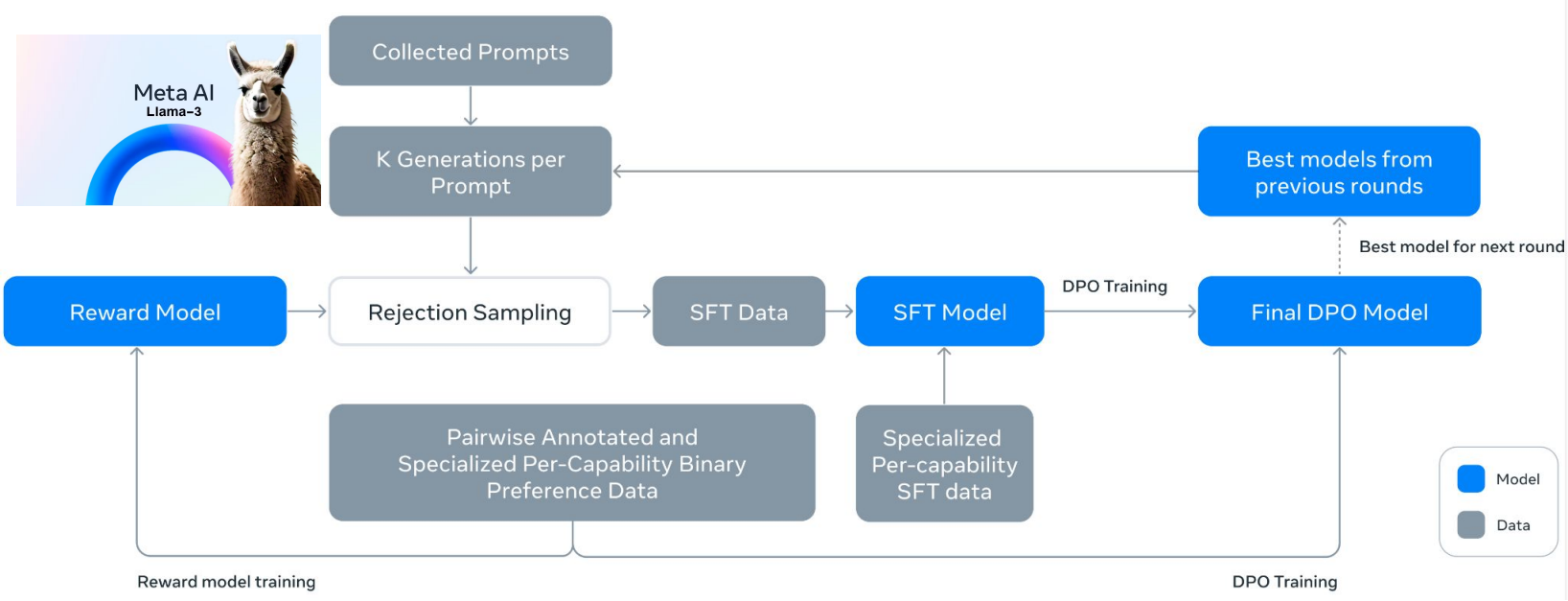
[Source]

Reinforcement Learning With Human Feedback (RLHF)



Proximal Policy Optimization (PPO), OpenAI, 2017

LLama 3 Herd


Meta AI
Llama-3
Collected Prompts
K Generations per Prompt
Best models from previous rounds
Best model for next round
Reward Model
Rejection Sampling
SFT Data
SFT Model
DPO Training
Final DPO Model
Pairwise Annotated and Specialized Per-Capability Binary Preference Data
Specialized Per-capability SFT data
Model
Data
Reward model training
DPO Training

PRM Ineffective DeepSeek

- Splitting up reasoning into discrete phases is difficult to implement
  - “Explicitly define fine-grain step in general reasoning”
- Difficult to tell which intermediate steps are correct
- Reward-hacking: model optimizes the PRM’s objective function but does not actually achieve the desired behavior

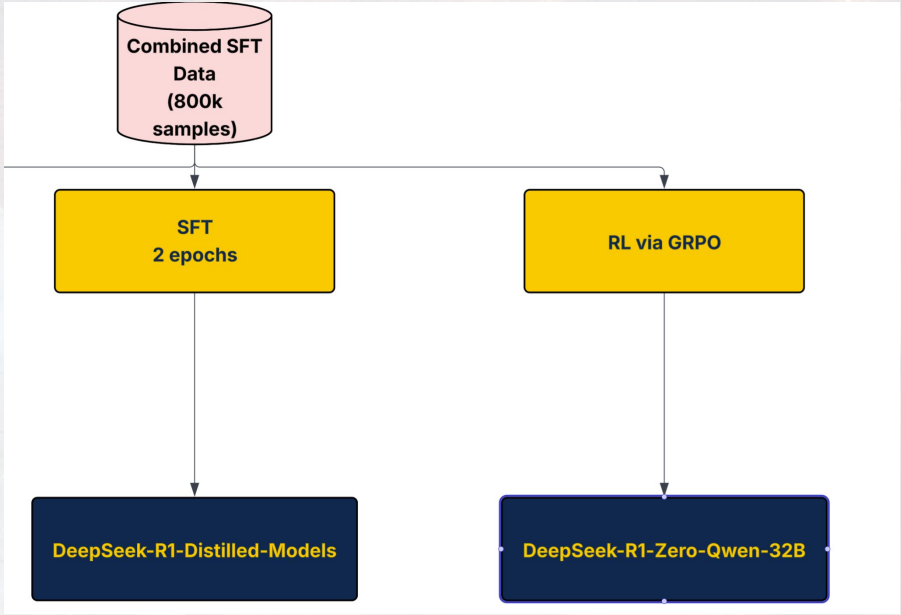
Motivation for Model Distillation

- Can we equip the smaller models with the knowledge of R1 via distillation?
- Does Large scale RL also improve smaller model reasoning performance?

Model Distillation

Dataset: 800k SFT data

1.



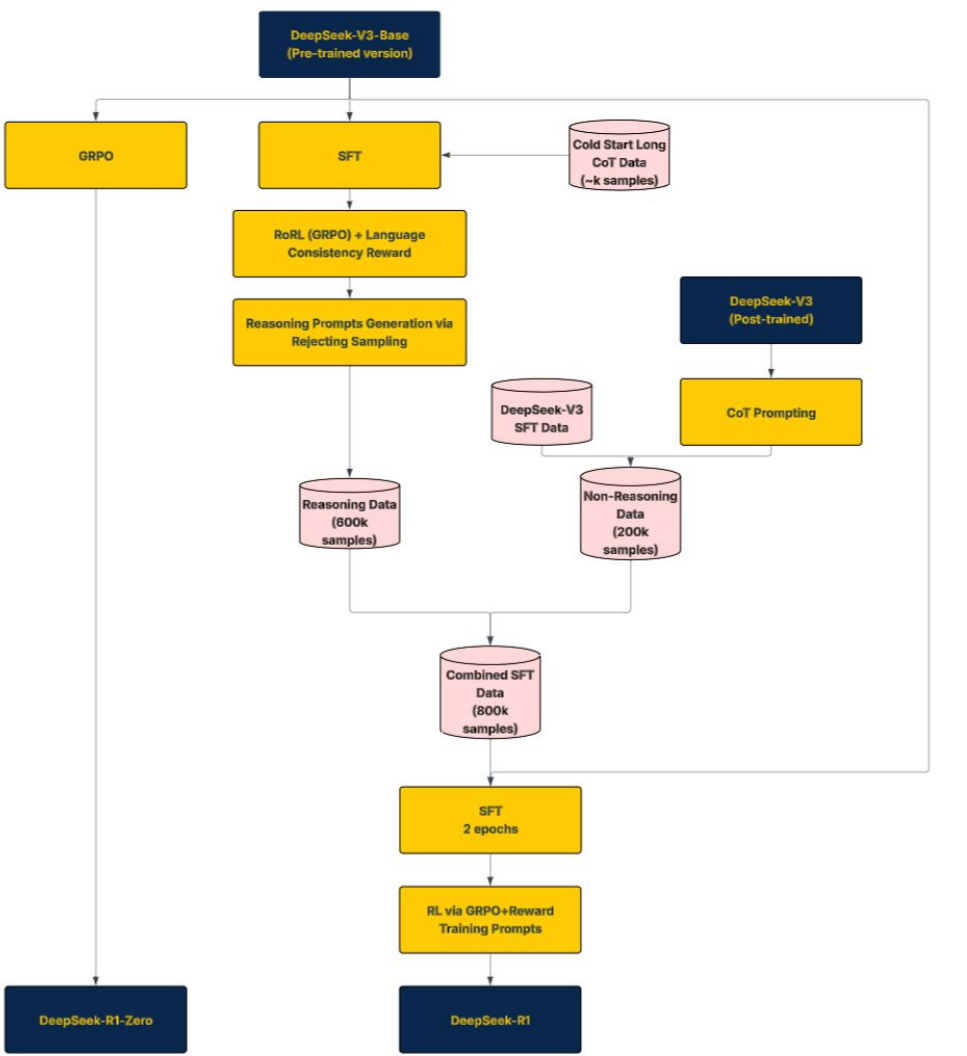
SFT Distillation Results

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	**1820**
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9	1316
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	**72.6**	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	**86.7**	**94.5**	**65.2**	**57.5**	1633

Large Scale RL Distillation Results

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCodeBench
	pass@1	cons@64	pass@1	pass@1	pass@1
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9
DeepSeek-R1-Zero-Qwen-32B	47.0	60.0	91.6	55.0	40.2
DeepSeek-R1-Distill-Qwen-32B	**72.6**	**83.3**	**94.3**	**62.1**	**57.2**

Not as effective as SFT distillation!



Credit: Reddit

The “Aha” moment

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a+x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a+x}} = x$, let's start by squaring both $\cdots$

$$\left(\sqrt{a - \sqrt{a+x}}\right)^2 = x^2 \implies a - \sqrt{a+x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

$\ldots$

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$

We started with the equation:

$$\sqrt{a - \sqrt{a+x}} = x$$

First, let's square both sides:

$$a - \sqrt{a+x} = x^2 \implies \sqrt{a+x} = a - x^2$$

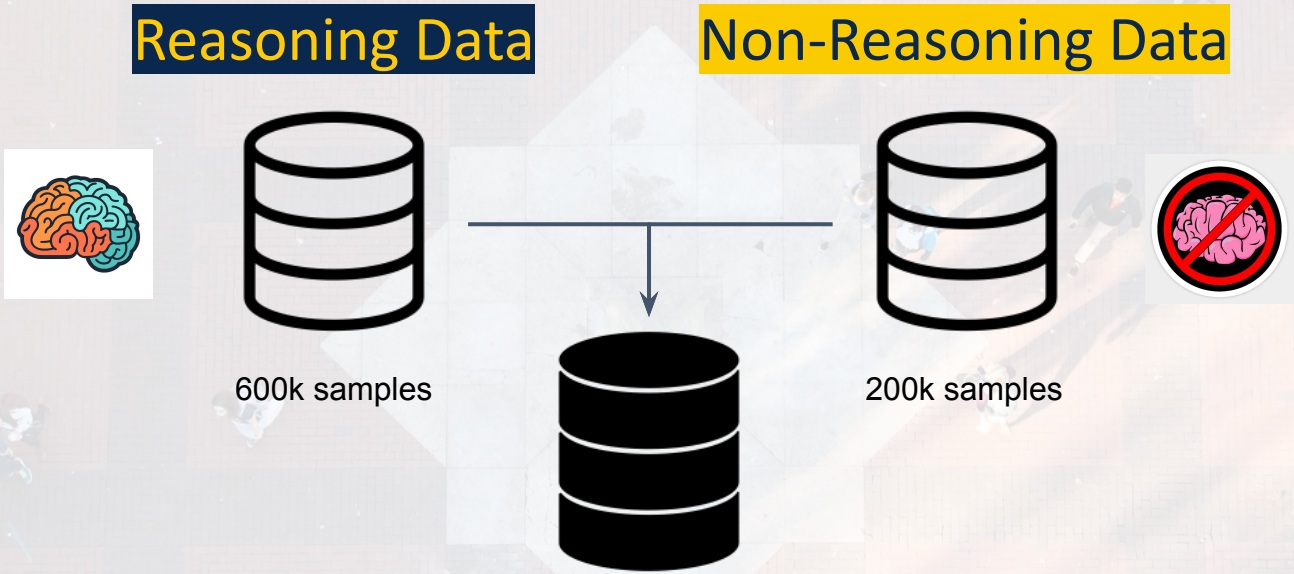
Next, I could square both sides again, treating the equation: $\cdots$

$\ldots$

DeepSeek R1 Motivation

- Can reasoning performance be further improved with a small SFT phase prior?
- How can we mitigate the readability issues and mix language issues?

High Quality Data Generation



Reasoning Data Generation: Textbook Creation

- Requires training a “teacher” model to generate a high quality “textbook”
- Teacher model (V3 Base) training
  - Cold Start SFT
  - Reason-Oriented Reinforcement Learning
  - Rejection Sampling

Cold Start SFT

- ~few thousand data samples
  - {prompt, long CoT output}
- Data collection methods used:
  - Few-shot prompting with long CoT
  - Direct prompting – prompt model to explicitly generate detailed answer
  - Collect readable outputs generated by R1-Zero
- Human Refinement
  - Annotators clean & refine all samples
- Enhances Teacher model readability & potential

Reasoning-oriented Reinforcement Learning

- Same rule-based reward system as R1-Zero
- Addition:
  - Language consistency reward
    - Proportion of target language words in CoT
- Observed slight decrease in performance
  - Due to more restrictive rules

Rejection Sampling

Rejection filters

- Incorrect reasoning task outputs
- Low-quality reasoning prompts e.g. bad formatting
- Use a judge (V3) to do quality filtering
  - Gets {prompt, Ground Truth output}
  - determines if Teacher Model output is satisfactory

Outcome

- **600k reasoning related training samples**

Non-Reasoning Data Generation

Teacher Model: DeepSeek-V3

Dataset breakdown:

- DeepSeek-V3 SFT dataset reused
- Prompt teacher model to generate CoT before answering

Outcome

- 200k non-reasoning training samples

Overall SFT

Dataset: 800k samples (600k reasoning + 200k non-reasoning)

Model: DeepSeek-V3-Base (Pre-trained, no SFT, no RL)

- Fine-tuned over 2 epochs only

RL for all Scenarios

Model: DeepSeek-V3-SFT-800k

Dataset: not specified

Reasoning data

- RL same as R1-Zero

Non-reasoning data

- Trained a reward model for non-rule-verifiable tasks
- Self-reward for open-ended prompts
- Ruled based reward system for rule-verifiable tasks
- Helpfulness reward
- Harmlessness reward